

# Deadline-Aware Spot Provisioning for Bag-of-Tasks Scheduling under Spot Hibernation

**Abstract**—Spot virtual machines are cheap but may be hibernated by the provider at any time, which threatens the deadline of a bag-of-tasks application. Burst-HADS addresses this by rescuing interrupted tasks onto burstable instances, buying makespan at a cost premium. We present R-BurstHADS, which adds deadline-aware preemptive and reactive spot provisioning on top of an otherwise unmodified Burst-HADS, and evaluate all three schedulers in one discrete-event simulator frozen before the final runs. The evaluation covers 80 cells — five hibernation scenarios, four bag sizes and four deadline factors — at 30 seeds each, giving 7,200 runs per launch-limit setting. Under the reference implementation’s provider launch limits, R-BurstHADS reduces makespan by 27.9 % against HADS and 12.7 % against Burst-HADS, while holding its cost premium over HADS to 12.9 % against Burst-HADS’s 21.3 % — a reduction of the premium by 8.4 points, about two fifths of it — and is 5.4 % cheaper than Burst-HADS. Without launch limits it is 17.8 % faster and 14.6 % cheaper than Burst-HADS, dominating it on both axes in 69 of 80 cells. We also report two negative results in full. Under limits R-BurstHADS is the only scheduler with more than a single missed task, 109 tasks in 4.3 % of runs against one and none, and all of them disappear when the same runs are repeated without limits. And our re-implemented baselines do not reproduce the published hibernation cost result: Burst-HADS costs 18.8 % more than HADS in our simulator against the 1.92 % reported. A fidelity audit accompanies the work, documenting 29 deviations from the published algorithms, of which 10 were corrected, 13 are deliberate design choices, one is a retained addition and five remain open.

**Index Terms**—cloud scheduling, spot instances, hibernation, burstable instances, bag-of-tasks, deadline-constrained scheduling, reproducibility

## I. INTRODUCTION

Infrastructure-as-a-service providers sell unused capacity at a steep discount on the understanding that they may reclaim it. On Amazon EC2 a spot instance can be *hibernated*: its memory and context are written to storage, execution stops, and the instance may or may not return. For a bag-of-tasks application with a deadline this is the central difficulty. Spot capacity is where the savings are, and it is also the capacity that disappears.

HADS [1] schedules such applications across spot and on-demand markets, checkpointing tasks and migrating them when an instance is reclaimed. Burst-HADS [2] extends it with burstable instances, which are kept available and can absorb a rescued task immediately at full speed by spending accumulated CPU credits, and with a work-stealing procedure that redistributes queued work when a machine becomes idle. The mechanism buys makespan, and it is not free: in our measurements under provider launch limits Burst-HADS

finishes 19.1 % sooner than HADS and pays 21.3 % more for it.<sup>1</sup>

That trade is the starting point of this work. Burst-HADS reacts to an interruption by finding somewhere to put the displaced task among the machines it already has; when the burstable pool is occupied, the only remaining destination is freshly rented on-demand capacity, the most expensive thing in the catalogue. A scheduler that instead anticipates interruptions, and that replaces lost capacity with more spot capacity rather than renting, should be able to keep the makespan benefit while giving back much of the premium.

## A. Contributions

- **R-BurstHADS**, a scheduler that adds two provisioning tiers around an unmodified Burst-HADS: a preemptive tier that buys replacement spot capacity for machines judged likely to be reclaimed, and a reactive tier that sizes new capacity from the remaining deadline slack when the published rescue attempts are exhausted (Section III).
- **A controlled comparison** of the three schedulers over 80 cells  $\times$  30 seeds under two account regimes — with and without the reference implementation’s launch limits — on the same 7,200 units of work (Sections VI and VII).
- **A fidelity audit**. We validate our re-implementations against the published results, report where they fail to reproduce them, rule out three candidate explanations by measurement, and publish a 29-row deviation register (Section V). Design decisions with more than one defensible answer were fixed by a rule committed to version control before the runs that decided them.
- **Negative results reported in full**, including the one regime in which our scheduler is the worst of the three (Sections VI-D and VIII).

## B. What this paper is not

It is not a reproduction of Teylo *et al.* [2]. Our HADS implementation lands within one to two percent of the published makespans on three of four workloads without hibernation, but our Burst-HADS does not reproduce either its published makespan reduction or, more importantly, its published hibernation cost behaviour. Section V reports this in detail, and every comparison in this paper is a comparison against our re-implementations as documented rather than against the published systems.

<sup>1</sup>Table IV, all-cells row.

TABLE I  
NOTATION

| symbol            | meaning                                      |
|-------------------|----------------------------------------------|
| $D$               | application deadline                         |
| $D_{\text{spot}}$ | $D$ minus the worst-case migration reserve   |
| $t$               | current simulation time                      |
| $s_v$             | per-core speed of machine $v$                |
| $k_v$             | usable execution slots (vCPUs) of $v$        |
| $\theta_v$        | whole-machine throughput, $s_v k_v$          |
| $r_v$             | price of $v$ per unit time                   |
| $\lambda_v$       | hibernation rate of $v$                      |
| $n_v$             | tasks the primary schedule placed on $v$     |
| $\bar{e}$         | mean task execution time                     |
| $\omega$          | instance start-up latency                    |
| $\Theta(A)$       | aggregate throughput of the active fleet $A$ |
| $W$               | remaining unfinished work                    |
| $P$               | set of machines provisioned by R-BurstHADS   |

## II. BACKGROUND

### A. Spot hibernation and the two baselines

HADS partitions a bag of tasks across a pool of spot and on-demand machines with a single-pass greedy primary scheduler, maintains a checkpoint for each running task, and on hibernation migrates the affected tasks by walking a fixed ladder: another machine already in the fleet first, then an on-demand machine already in the fleet, and finally a newly launched on-demand machine.

Burst-HADS replaces the primary scheduler with an iterated local search that minimises a weighted combination of cost and makespan, and inserts burstable instances into the recovery ladder. A burstable machine holds exactly one task at a time. Rescued onto one, a task runs in *burst mode* at full speed and spends CPU credits; picked up by work stealing, it runs in *baseline mode* at a fraction of full speed and earns them.

### B. Burstable economics depend on the catalogue

Whether rescuing onto a burstable is a saving is a property of the price list, not of the algorithm. In the catalogue used by [2] a burst-mode task pays  $0.83\times$  what the same task pays running alone on a newly launched on-demand machine, which is the alternative the recovery ladder would otherwise take. In a catalogue built from current Amazon EC2 prices the same comparison is  $1.15\times$ : the burstable instance is *dearer* per unit of work than the on-demand machine it is meant to displace.<sup>2</sup> We report results on both catalogues and treat this as a finding rather than a parameter choice.

## III. R-BURSTHADS

R-BurstHADS inherits Burst-HADS in full — the same primary scheduler, the same Algorithm 4 recovery attempts, the same work stealing — and wraps two provisioning tiers around them. Nothing in the inherited path is modified, so a correction to the baseline’s fidelity is inherited automatically. Notation is given in Table I.

<sup>2</sup>Table II; full price table in the results pack [T15].

### Algorithm 1 R-BurstHADS: destination selection on hibernation

**Require:** task  $\tau$ , time  $t$ , deadline  $D$

#### Tier 0 — preemptively provisioned capacity

```

1: for all  $v \in P$  alive do
2:   if CHECKMIGRATION( $\tau, v, t, D$ ) then return  $v$ 
3:   end if
4: end for
```

#### Tiers 1–2 — Burst-HADS, unmodified

```

5:  $v \leftarrow \text{PAPERMIGRATION}(\tau, t)$   $\triangleright$  idle burstable, then any
   active machine
6: if  $v \neq \perp$  then return  $v$ 
7: end if
```

#### Tier 3 — reactive provisioning

```

8:  $v \leftarrow \text{PROVISIONONEMORE}(\tau, t)$   $\triangleright$  Eq. (3)
9: if  $v \neq \perp$  then return  $v$ 
10: end if
```

#### Tier 4 — Burst-HADS, unmodified

```

11: return ONDEMANDFALLBACK( $\tau, t$ )
```

### A. Tier 0: preemptive replacement

After the primary schedule is built, each spot machine is examined once. A replacement is bought when the expected cost of reacting to its loss exceeds the cost of anticipating it:

$$(1 - e^{-\lambda_v D}) C_{\text{react}}(v) > C_{\text{proact}}, \quad (1)$$

where the reactive cost is the extra paid to finish  $v$ ’s work on rented on-demand capacity instead of spot,

$$C_{\text{react}}(v) = n_v \bar{e} \left( \frac{r_{\text{od}}}{\theta_{\text{od}}} - \frac{r_s}{\theta_s} \right), \quad (2)$$

and the proactive cost is the replacement’s idle boot period,  $C_{\text{proact}} = \omega r_s + \epsilon$ . Both rate terms are divided by *whole-machine* throughput  $\theta = sk$ , not by per-core speed; dividing by speed alone prices a four-core machine as though it retired one core’s worth of work.

The replacement’s instance type is resolved from the scheduler’s own pool, never from a private constant, so R-BurstHADS can only obtain a type the baselines’ planners could also have selected. Among types that are more likely than not to survive to the deadline, it takes the best throughput per unit price, breaking exact ties toward the larger machine.

### B. Tier 3: deadline-sized reactive provisioning

When an interruption occurs and the inherited attempts are exhausted, the shortfall is sized from what is left of the deadline. With  $T = D - t - \omega$  the usable time after a new machine boots, the number of additional machines is

$$n_{\text{extra}} = \left\lceil \frac{W/T - \Theta(A)}{\theta} \right\rceil. \quad (3)$$

Spot capacity is bought when the remaining slack comfortably exceeds the boot latency and the task can still finish in time on the new machine; otherwise a burstable instance is taken, which is usable immediately.

**Algorithm 2** PROVISIONONEMORE**Require:** task  $\tau$ , time  $t$ 

```

1:  $slack \leftarrow D - t$ 
2: if  $slack > 2\omega$  then
3:    $f \leftarrow t + \omega + \tau.remaining/s_{tpl}$ 
4:   if  $f \leq D$  and  $D - f > \tau.e/s_{tpl}$  then
5:      $v \leftarrow \text{LAUNCHSPOT}(\text{ready at } t + \omega); P \leftarrow P \cup \{v\}$ 
6:     return  $v$ 
7:   end if
8: end if
9: if  $t + \tau.remaining/s_b \leq D$  then
10:   $v \leftarrow \text{LAUNCHBURSTABLE}(\text{ready at } t); P \leftarrow P \cup \{v\}$ 
11:  return  $v$ 
12: end if
13: return  $\perp$ 

```

TABLE II

PRICE PER UNIT OF WORK. “1 TASK” IS WHAT A TASK PAYS RUNNING ALONE ON THE MACHINE, WHICH IS THE COMPARISON THE RECOVERY LADDER MAKES.

| catalogue | type (market)      | \$/h   | \$/unit full | \$/unit, 1 task |
|-----------|--------------------|--------|--------------|-----------------|
| sweep     | c5.large (spot)    | 0.0306 | 0.00765      | 0.01530         |
| sweep     | c5.xlarge (spot)   | 0.0612 | 0.00765      | 0.03060         |
| sweep     | m5.xlarge (spot)   | 0.0700 | 0.01029      | 0.04118         |
| sweep     | c5.large (on-dem.) | 0.0850 | 0.02125      | 0.04250         |
| sweep     | t3.large (burst.)  | 0.0832 | 0.04894      | 0.04894         |
| valid.    | c4.large (spot)    | 0.0366 | 0.01830      | 0.03660         |
| valid.    | c4.large (on-dem.) | 0.1000 | 0.05000      | 0.10000         |
| valid.    | t3.large (burst.)  | 0.0832 | 0.08320      | 0.08320         |

The guard on line 4 of Algorithm 2 is the spare-time rule of [2, §3.4]: a spot destination must leave more slack than the longest task placed on it, so that a second interruption can still be absorbed. Tier 0 obtains the same rule by calling the inherited CHECKMIGRATION; tier 3 states it explicitly because the machine has not yet booted and the inherited estimator has no notion of boot latency.

## IV. METHOD

## A. Simulator and freeze

All three schedulers run in one discrete-event simulator. The simulator was frozen before the final runs at a tagged commit, and every result in this paper comes from that code; defects found after the freeze are recorded in the deviation register rather than fixed. Every result file is committed, and the results pack that generates the tables below records each file’s hash and the commit it came from.

## B. Catalogues

Two instance catalogues are used: a *sweep* catalogue at current EC2 prices for the main comparison, and a *validation* catalogue reproducing Table 3 of [2] for the fidelity audit. Each has three spot types, three on-demand types and one burstable type. Key economics are in Table II.

## C. Experiment grid

The grid crosses the five hibernation scenarios of [2, Table 9] — sc1 ( $k_h, k_r$ ) = (1, 0), sc2 (5, 0), sc3 (1, 5), sc4 (5, 5), sc5 (3, 2.5), with per-machine rates  $\lambda_h = k_h/D$  and  $\lambda_r = k_r/D$  — with bag sizes  $n \in \{50, 100, 200, 300\}$  and deadline factors  $DF \in \{0.25, 0.5, 1.0, 2.0\}$ , at 30 seeds per cell. That is 80 cells and 7,200 runs per limit setting. The deadline is  $D = DF \cdot \kappa \cdot M^*(n)$  with a floor of 339.7s; the floor binds at  $DF = 0.25$  for  $n = 100$ .

Deadline factor is swept rather than fixed because both baselines only escalate when the deadline is threatened, so a single deadline setting measures the slack granted as much as the scheduling policy. Reporting across  $DF$  makes the constant in  $D$  non-load-bearing.

## D. Launch limits

The reference implementation limits an account to five launches per instance type per market and twenty per market, with burstable instances counted against the on-demand allowance. An uncapped pool lets any deadline be bought and flattens deadline satisfaction into a metric that cannot discriminate, so we adopt the limits and additionally report the same 7,200 units without them. The starting pool holds three spot machines per type, below the limit of five, so the spot limit binds only on capacity R-BurstHADS launches itself. Both regimes are reported throughout, leading with limits on.

## E. Metrics

Cost is billed per second from launch to shutdown or hibernation, following [2, §3.1]: a machine’s meter opens when a scheduler launches it, whether or not it receives work. A cell is a (scenario,  $n$ ,  $DF$ ) triple; a reported percentage is the ratio of cell means, averaged over cells. Confidence intervals for R-BurstHADS against Burst-HADS are paired per seed at 95%. Deadline satisfaction is reported as two measures — missed tasks, and the share of runs with any miss — because they order the schedulers differently. Runs whose primary scheduler finds no placement within  $D$  and the limits are reported as *infeasible* and excluded from cross-scheduler means, with their cause.

## F. Decision discipline

Every change to a baseline was first measured as a counterfactual patch and adopted only when the adopted code reproduced the measured variant row for row: 1,440 of 1,440 validation rows and 2,400 of 2,400 sweep rows. Where two principled options existed, the choice rule was committed to version control before the runs that would decide it.

## V. FIDELITY AUDIT

## A. What reproduces

Without hibernation, our HADS lands close to the published makespans: 2330s against 2290s on J60, 2350 against 2295 on J80, 2356 against 2332 on J100, and 2365 against 2580 on ED200.

TABLE III

VALIDATION AGAINST [2, TABLE 9], 20 CELLS, SAME DEFINITION FOR BOTH. PREMIUM IS A SCHEDULER'S COST UNDER HIBERNATION AGAINST ITS OWN COST WITHOUT.

| quantity                | ours, limits off | limits on | [2]     |
|-------------------------|------------------|-----------|---------|
| Burst-HADS cost vs HADS | +18.80 %         | +18.66 %  | +1.92 % |
| Makespan reduction      | 22.18 %          | 22.18 %   | 25.87 % |
| J60 only                | 35.43 %          | 35.43 %   | 40.10 % |
| ED200 only              | 5.28 %           | 5.25 %    | 10.24 % |
| HADS premium            | +105 %           | +105 %    | +95 %   |
| Burst-HADS premium      | +40 %            | +40 %     | +25 %   |

### B. What does not

Our Burst-HADS does not. Its makespan reduction against HADS without hibernation is  $-70.9/-59.8/-50.7/-0.1\%$  on the four workloads against the published  $-44.4/-42.1/-28.8/-11.8\%$ : far stronger than published on the smaller jobs and flat on the largest. Its J60 cost, however, is within two percent of the published figure.

Under hibernation neither published aggregate is reproduced (Table III). The headline discrepancy is cost: Burst-HADS costs 18.8% more than HADS in our simulator against the published 1.92%. The two regimes agree to within 0.14 points, so this is not an artefact of the launch limits.

### C. Where the gap is, and what it is not

The gap is not shared between the schedulers. Hibernation raises HADS's cost by 105% against a published 95%, which is close; it raises Burst-HADS's by 40% against a published 25%. Burst-HADS's premium is the discrepancy.

Three candidate explanations were tested and rejected by measurement. The *aggregation* is not at fault: alternative weightings of our runs give +36.6 to +42.4% against our headline +38.7% at the time of the test, all far from +1.9%. The *billing rule* is not at fault: no rule closes the gap, with 900s quantisation giving +30.1%, billing from launch +40.4%, the published rule +42.5%, and an earlier rule +22.9% at the price of wrecking the no-hibernation validation. *Storage charges during hibernation* move the figure from +38.7 to +37.5%.

What remains is exposure. Our runs suffer between 1.3 and 3.6 times the hibernations per run that the published table implies, because our schedulers deploy most of the spot pool and hibernation is applied per machine. The pool size on which those counts depend is not stated in [2], and no value we tried reproduces its Table 7 and Table 9 together. We record this as a finding about the published description rather than as a parameter to tune.

### D. The audit moved the numbers

Four baseline corrections adopted during the audit moved the validation toward the published results: the cost change from +38.7 to +18.8%, the makespan reduction from 14.7 to 22.2%, and Burst-HADS's premium from +94 to +40%. They did not close the gap.

fig1\_headline

Fig. 1. Makespan and cost against HADS under launch limits, over the 75 cells feasible for every scheduler in every seed. R-BurstHADS roughly halves Burst-HADS's cost premium while finishing sooner. Over all 80 cells with available seeds the makespan figures are 26.4% against HADS and 11.9% against Burst-HADS.

### E. Deviation register

Twenty-nine deviations from the published algorithms are recorded with their measured effect: 10 corrected, 13 deliberate design choices, one retained addition and five open, one of which is unmeasured. The register is reproduced in Appendix A.

One entry deserves the main text. Our Burst-HADS keeps an improvement guard in its proactive burstable allocation which the published description does not have, and the guard favours the comparison reported here: with it, R-BurstHADS is 5.3% cheaper than Burst-HADS with 34 of 55 cells dominated; without it, 5.8% dearer with 14 of 55. It was kept by the pre-registered rule, because removing it turned 85 clean Burst-HADS runs and 92 clean R-BurstHADS runs into runs with a missed task, against none and three in the other direction.

## VI. RESULTS UNDER PROVIDER LAUNCH LIMITS

### A. Headline

Over the 75 cells in which every scheduler is feasible in every seed (Table IV, Fig. 1), Burst-HADS reduces makespan by 19.1% against HADS at 21.3% more cost. R-BurstHADS reduces it by 27.9% at 12.9% more cost: it cuts the premium by 8.4 points, about two fifths of it, and is simultaneously 12.7% faster and 5.4% cheaper than Burst-HADS, dominating it on both axes in 44 of the 75 cells. Paired per-seed intervals put it significantly faster than Burst-HADS in 45 cells and slower in 9, significantly cheaper in 29 and dearer in 7.

### B. The advantage needs deadline slack

Figures 2 and 3 show that the advantage is concentrated where the deadline is loose. Against Burst-HADS, R-BurstHADS is 28.9% faster and 14.9% cheaper at  $DF = 2.0$ , dominating 17 of 20 cells; 15.9% faster and 4.3% cheaper at  $DF = 1.0$ ; and essentially tied at  $DF = 0.5$ , at 1.4% faster and equal on cost, with 7 cells where it is significantly slower

TABLE IV  
RESULTS UNDER LAUNCH LIMITS. B AND R ARE BURST-HADS AND R-BURSTHADS; H IS HADS. PERCENTAGES ARE RATIOS OF CELL MEANS AVERAGED OVER CELLS.

| group     | cells | B mk<br>vs H | B \$<br>vs H | R mk<br>vs H | R \$<br>vs H | R mk<br>vs B | R \$<br>vs B |
|-----------|-------|--------------|--------------|--------------|--------------|--------------|--------------|
| all       | 75    | -19.1        | +21.3        | -27.9        | +12.9        | -12.7        | -5.4         |
| $DF=0.25$ | 15    | -7.3         | +9.7         | -8.9         | +8.4         | -1.8         | -1.3         |
| $DF=0.5$  | 20    | -6.2         | +18.6        | -7.4         | +18.6        | -1.4         | -0.0         |
| $DF=1.0$  | 20    | -13.1        | +24.2        | -27.0        | +16.6        | -15.9        | -4.3         |
| $DF=2.0$  | 20    | -46.6        | +30.0        | -63.6        | +7.1         | -28.9        | -14.9        |
| sc1       | 15    | -27.1        | +36.3        | -39.2        | +19.8        | -18.9        | -9.8         |
| sc2       | 15    | -13.0        | +20.2        | -18.2        | +18.8        | -7.7         | -1.3         |
| sc3       | 15    | -25.5        | +24.8        | -37.6        | +10.4        | -17.4        | -9.5         |
| sc4       | 15    | -12.7        | +12.6        | -18.8        | +10.2        | -7.5         | -1.0         |
| sc5       | 15    | -16.9        | +12.8        | -25.8        | +5.5         | -12.0        | -5.3         |
| $n=50$    | 20    | -11.7        | +32.4        | -28.7        | +11.5        | -19.2        | -12.3        |
| $n=100$   | 15    | -18.4        | +28.1        | -31.2        | +16.5        | -18.9        | -8.6         |
| $n=200$   | 20    | -21.8        | +14.5        | -25.6        | +11.4        | -7.7         | -2.6         |
| $n=300$   | 20    | -24.2        | +12.1        | -27.1        | +13.2        | -6.5         | +1.1         |
| all 80    | 80    | -18.0        | +20.0        | -26.4        | +12.2        | -11.9        | -5.0         |

fig2\_cost\_vs\_df

Fig. 2. Cost against HADS as a function of deadline slack, under launch limits. The two schedulers are indistinguishable at  $DF=0.5$ , both at +18.6%, and separate as slack grows. R-BurstHADS is never cheaper than HADS in this regime.

and 6 where it is significantly dearer. At  $DF = 0.25$  the difference is 1.8% and 1.3%.

The reason is visible in HADS's own behaviour: it finishes at 93% of the deadline on average and 99% at  $DF = 0.25$ . Both baselines escalate only when the deadline is threatened, so when it is tight they spend whatever is needed and the three schedulers converge; when it is loose they trade makespan for cost and R-BurstHADS, whose provisioning is triggered by interruption risk rather than by deadline pressure alone, does not.

### C. By scenario and by bag size

Against Burst-HADS the margin is widest in the low-interruption scenarios — 18.9% faster and 9.8% cheaper in sc1, 17.4% and 9.5% in sc3 — and narrowest in the high-interruption ones, 7.5% and 1.0% in sc4 and 7.7% and

fig3\_makespan\_vs\_df

Fig. 3. Makespan against HADS as a function of deadline slack, under launch limits. Both baselines finish close to the deadline wherever they can buy their way there, so the gap widens with slack.

fig4\_scenarios

Fig. 4. Cost against HADS by hibernation scenario, under launch limits. sc1 is where R-BurstHADS costs the most relative to HADS and also where it beats Burst-HADS by the widest margin.

1.3% in sc2, each with three cells where R-BurstHADS is significantly dearer. Fig. 4 shows the complementary view against HADS: sc1 is both where R-BurstHADS pays the largest premium over HADS, 19.8%, and where it beats Burst-HADS by the most.

The advantage also narrows with scale (Fig. 5). At  $n = 50$  R-BurstHADS is 19.2% faster and 12.3% cheaper than Burst-HADS, dominating 17 of 20 cells. At  $n = 300$  it is 6.5% faster and 1.1% dearer, dominating only 5 of 20. The reversal is weak — three cells significantly cheaper against two significantly dearer — and it does not appear without launch limits, where the same comparison is 12.8% cheaper. We read it as the launch ceiling binding hardest where the work is largest.

### D. Deadline misses

This is where our scheduler is the worst of the three (Table V, Fig. 6). Under launch limits HADS misses no task, Burst-HADS misses one task in one run, and R-BurstHADS

fig5\_cost\_vs\_n

Fig. 5. Cost against HADS by bag size, under launch limits. The cost advantage over Burst-HADS narrows as the bag grows and reverses at  $n=300$ , where R-BurstHADS is 1.1 % dearer. Without limits there is no reversal.

fig6\_missed\_tasks

Fig. 6. Missed tasks with and without launch limits. R-BurstHADS is the only scheduler with more than a single missed task under limits, and every miss disappears when the same runs are repeated without them.

TABLE V  
DEADLINE MISSES, BOTH MEASURES, FEASIBLE RUNS ONLY.

| limits | scheduler   | runs | runs w/ miss | share | missed tasks |
|--------|-------------|------|--------------|-------|--------------|
| on     | HADS        | 2390 | 0            | 0.0 % | 0            |
| on     | Burst-HADS  | 2400 | 1            | 0.0 % | 1            |
| on     | R-BurstHADS | 2400 | 104          | 4.3 % | 109          |
| off    | HADS        | 2400 | 0            | 0.0 % | 0            |
| off    | Burst-HADS  | 2400 | 0            | 0.0 % | 0            |
| off    | R-BurstHADS | 2400 | 0            | 0.0 % | 0            |

misses 109 tasks in 104 of 2,400 runs — 4.3 % of runs, and 0.028 % of all tasks. Eighty of the 109 are at  $DF = 0.5$ . The worst cells are sc4 and sc5 at  $n = 300$ ,  $DF = 0.5$ , with 15 missed tasks each, half a task per run.

The evidence that the launch ceiling is responsible is the matched counterfactual, not the limit flag. Repeating the same 7,200 units without limits removes every missed task. It is true that all 104 missing runs had reached a launch limit, most often the c5.xlarge spot limit in 103 of them; but so had 93.5 % of

TABLE VI  
RESULTS WITHOUT LAUNCH LIMITS, 80 CELLS.

| group     | B mk<br>vs H | B \$<br>vs H | R mk<br>vs H | R \$<br>vs H | R mk<br>vs B | R \$<br>vs B |
|-----------|--------------|--------------|--------------|--------------|--------------|--------------|
| all       | -18.1        | +21.9        | -31.0        | +2.6         | -17.8        | -14.6        |
| $DF=0.25$ | -6.0         | +12.6        | -8.2         | +8.7         | -2.4         | -3.4         |
| $DF=0.5$  | -6.4         | +20.5        | -13.2        | -2.1         | -7.2         | -18.2        |
| $DF=1.0$  | -13.1        | +24.6        | -34.2        | +2.3         | -24.0        | -16.8        |
| $DF=2.0$  | -46.6        | +30.0        | -68.5        | +1.4         | -37.7        | -19.9        |

R-BurstHADS’s runs *without* a miss, so reaching a limit does not by itself identify the failures.

#### E. Infeasible runs

Ten runs of 7,200 have no feasible primary schedule, all of them HADS, at  $n = 100$  and  $DF = 0.25$  where the deadline sits on its 339.7s floor, in seeds 1 and 24 of every scenario. Burst-HADS and R-BurstHADS are feasible in all 2,400 runs each. Because a cross-scheduler cell mean is undefined when one scheduler has no run, those five cells are excluded from Table IV’s main rows and reported separately; the last row gives the all-80-cell figures over available seeds. Without limits no run is infeasible.

#### VII. RESULTS WITHOUT LAUNCH LIMITS

Repeating the same 7,200 units with the account limits removed isolates how much of R-BurstHADS’s behaviour depends on being allowed to launch (Table VI). All 80 cells are feasible and no scheduler misses a deadline.

R-BurstHADS is then 17.8 % faster and 14.6 % cheaper than Burst-HADS, dominating it in 69 of 80 cells, significantly faster in 70 and slower in none, significantly cheaper in 59 and dearer in 4. Against HADS it is 31.0 % faster at a 2.6 % premium.

The contrast identifies what the advantage runs on. Burst-HADS barely notices the regime change, moving from -19.1 %/ +21.3 % to -18.1 %/ +21.9 % against HADS, because it never launches enough to reach a limit. R-BurstHADS changes a great deal: its cost lead over Burst-HADS grows from 5.4 % to 14.6 %, its premium over HADS falls from 12.9 % to 2.6 %, and its deadline misses vanish. At  $DF = 0.5$ , where the two are tied under limits, removing them makes R-BurstHADS 7.2 % faster and 18.2 % cheaper in 19 of 20 cells.

Headroom, in other words, is the resource this scheduler trades on. Where the provider grants it the method is strong; where it does not, most of the cost advantage and all of the reliability advantage go away.

#### VIII. THREATS TO VALIDITY AND LIMITATIONS

**This is not a reproduction.** The published hibernation cost result is not reproduced, +18.8 % against +1.92 %, and neither are the published Burst-HADS makespans without hibernation. Every comparison here is against our re-implementations as documented in the deviation register, and the residual gap is concentrated in Burst-HADS’s hibernation premium, which is  $1.6\times$  the published value. Our Burst-HADS is therefore a more

expensive baseline under hibernation than the published one, and R-BurstHADS’s cost advantage over it is flattered by an amount this study has not bounded.

**The advantage depends on launch headroom.** Under limits R-BurstHADS is the only scheduler with more than a single missed task, and its cost lead over Burst-HADS falls from 14.6 % to 5.4 %.

**A retained addition favours the comparison.** The proactive-burstable improvement guard is ours, not the published algorithm’s, and removing it turns R-BurstHADS’s 5.3 % cost lead into a 5.8 % deficit.

**The sweep catalogue penalises the baseline’s mechanism.** At current prices a burst-mode rescue costs  $1.15\times$  the on-demand alternative, against  $0.83\times$  in the published catalogue, so Burst-HADS’s central mechanism is a cost penalty in the main comparison.

**Per-type risk is never exercised.** The scenarios hibernate every spot machine at the same rate, so the per-type hibernation differential that Eq. (1) reasons about is never instantiated. No result in this paper is attributed to risk awareness.

**Five deviations remain open**, one of them unmeasured.

**Scope.** One simulator, synthetic workloads, 30 seeds per cell.

## IX. FUTURE WORK

Four items follow directly. The spot pool size is not stated in [2] and its hibernation counts depend on it; a sensitivity study over pool size would bound the validation gap. The provisioning and replacement-utilisation diagnostics predate the freeze and should be re-run on the frozen code. The one unmeasured deviation should be measured. And heterogeneous per-type hibernation rates would, for the first time, exercise the risk reasoning in Eq. (1).

## X. CONCLUSION

Burst-HADS buys makespan with cost. Across 80 cells under provider launch limits it finishes 19.1 % sooner than HADS and pays 21.3 % more. R-BurstHADS, which adds preemptive and deadline-sized reactive spot provisioning around an otherwise unmodified Burst-HADS, finishes 27.9 % sooner than HADS at a 12.9 % premium — 12.7 % faster and 5.4 % cheaper than Burst-HADS, and dominant on both axes in 44 of 75 cells. Without launch limits the margin widens to 17.8 % and 14.6 %, in 69 of 80 cells.

Two results cut the other way and we report them as prominently as the first. Under launch limits R-BurstHADS is the only scheduler with more than a single missed task, 109 in 4.3 % of runs, and all of them disappear once the limits are removed: the method depends on provisioning headroom that a provider may not grant. And our re-implemented baselines do not reproduce the published hibernation cost result, which we document, decompose and leave open rather than resolve. The accompanying deviation register is offered as part of the contribution: a comparison whose disagreements with its own sources are stated is worth more than one whose are not.

## REFERENCES

- [1] L. Teylo *et al.*, “A hibernation aware dynamic scheduler for cloud environments,” *Cluster Computing*, 2021.
- [2] L. Teylo *et al.*, “Scheduling bag-of-tasks in clouds using spot and burstable virtual machines,” *IEEE Transactions on Cloud Computing*, 2023.

## APPENDIX A DEVIATION REGISTER

The full register, with the measured effect of each entry, is reproduced from `DEVIATIONS.md`.

## APPENDIX B PER-CELL RESULTS

Per-cell means and 95 % confidence intervals for makespan and cost, all three schedulers, both limit regimes; and the infeasible runs with their cause.

## APPENDIX C VALIDATION DETAIL

Comparison against the published Table 7 and Table 9, cell by cell; the ruled-out causes of the cost gap; and the trajectory of the audit’s corrections.

## APPENDIX D REPRODUCIBILITY

Frozen tag and code fingerprint, the results pack with per-file hashes and commits, the generating script, and the pre-registered decision rule.